

LCNSM — Liability Condition State Module

LCNSM — Liability Condition State Module

Architecture: LIGA™ — Liability Governance Architecture
Parent Standard: Liability Conditions Standard (LCOS)
Category: Governance & Enforcement
Subcategory: Liability Governance
Type: Core Module
Governed Space: Liability Condition State
Module ID: LIGA-LCOS-LCNSM-001
Origin ID: OOF.CM.LIGA.05.03.01
Version: 1.0
Status: Canonical · Open Standard
Effective Date: 22 September 2026
Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem
AI-Readable: Yes
Authority: OOF® — OriginOpen® Foundation
Protection: MIP® — Methodological Intellectual Property
Canonical Language: English (UCL™)

Canonical Definition

The Liability Condition State Module (LCNSM) governs how governed inputs are evaluated against the Governed Liability Condition Requirement Set to determine, represent, distinguish, and preserve the state of each Liability Condition at the materially relevant lifecycle moment.

LCNSM determines whether an applicable Liability Condition is Satisfied, Not Satisfied, Partially Satisfied, Failed, Present, Absent, Pending, Unknown, Uncertain, Disputed, or otherwise represented through an explicitly governed Condition State.

LCNSM determines Condition State.

It does not determine Liability-Relevant Connection, Liability Attachment, or Liability.

It governs the transition:

Governed Liability Condition Requirement Set → Evaluation Input Association → Requirement Evaluation → Condition State Determination → State Confidence & Uncertainty Governance → Governed Liability Condition

State Set

Its foundational distinction is:

Condition State ≠ Liability Determination.

Operational Role

The operational role of LCNSM is:

to determine the governed state of each Liability Condition by evaluating relevant governed inputs against its explicit Condition Requirement.

LCNIM determines:

Which Conditions require governance?

LCNRM determines:

What does each Condition require?

LCNSM determines:

What is the governed state of each Condition?

LCNSM does not determine what that Condition State ultimately means for Liability.

Position in the Parent Standard Flow

The internal LCOS progression is:

Liability Condition Identification → Liability Condition Requirement → Liability Condition State → Liability Condition Exception → Liability Condition Continuity LCNSM governs:

Liability Condition State

Its principal input is:

Governed Liability Condition Requirement Set

Its principal output is:

Governed Liability Condition State Set

Its canonical handoff is:

Governed Liability Condition State Set → Liability Condition Exception Governance

Central Operational Question

What is the governed state of each Liability Condition when the relevant inputs are evaluated against its defined Requirement?

Governed Space

LCNSM governs:

Condition State Determination, Evaluation Input Association, Requirement-to-State Evaluation, Satisfied Condition State, Not Satisfied Condition State, Failed Condition State, Partially Satisfied Condition State, Present Condition State, Absent Condition State, Pending Condition State, Unknown Condition State, Uncertain Condition State, Disputed Condition State, Threshold State, Temporal Condition State, Positive Condition State, Negative Condition State, Subject-Specific Condition State, Relationship-Specific Condition State, Object-Specific Condition State, Basis-Specific Condition State, Compound Condition State, State Confidence, State Conflict, State Correction, State Reassessment, and State Sufficiency.

Liability Condition State

A Liability Condition State is the governed result produced by evaluating relevant inputs against an explicit Liability Condition Requirement.

The State describes the Condition.

It does not describe Liability itself.

Therefore:

Liability Condition State ≠ Liability State.

Requirement Before State

A Condition State must be evaluated against a defined Requirement.

The governed sequence is:

Condition → Requirement → Relevant Input → Evaluation → Condition State not:

Observed Outcome → Condition State Therefore:

Condition State without Requirement ≠ Governed Condition State.

Evaluation Input

An Evaluation Input is a governed fact, state, record, measurement, evidence-supported representation, validated output, or other relevant input used to evaluate a Condition Requirement.

Inputs may originate from:

LIGA™, ORA™, GOA™, AGA™, HAGA™, ADIT®, VALIDOS™, OBIDENITY®, AIG®, ASGA™, or another relevant governance domain.

The source should remain traceable.

Input and State

The existence of an Evaluation Input does not itself establish the Condition State.

The input must be evaluated against the Requirement.

Therefore:

Evaluation Input ≠ Condition State.

Input Relevance

Only inputs materially relevant to the Requirement should affect the State determination.

The system should preserve:

which input was used, why it was relevant, which Requirement component it addressed, and the applicable temporal state.

Multiple Inputs

A Condition may require multiple Evaluation Inputs.

Those inputs may:

support one another, address different Requirement components, conflict, or leave gaps.

Therefore:

Multiple Inputs ≠ Automatic Condition Certainty.

Missing Input

Required input may be unavailable.

The absence of input should not automatically produce a failed Condition.

Therefore:

Missing Input ≠ Failed Condition by default.

The appropriate State may instead be:

Unknown, Pending, or another explicitly governed unresolved state.

Evidence and Condition State

ADIT® may govern evidence supporting an Evaluation Input.

LCNSM governs the Condition State resulting from evaluation.

Therefore:

Evidence ≠ Condition State.

Evidence may support a State without becoming the State.

Validated Input

VALIDOS™ may validate an input used in State evaluation.

Validation strengthens confidence in the input where applicable.

However:

Validated Input ≠ Satisfied Condition by default.

The validated input must still satisfy the Requirement.

Operational Reality Input

ORA™ may provide actual operational states.

For example:

who acted, what occurred, which system operated, who exercised control, or what operational state existed.

LCNSM may use those states as Evaluation Inputs.

Therefore:

Operational Reality ≠ Condition State by itself.

Condition State Vocabulary

A conforming implementation should support explicit states appropriate to the Condition structure.

These may include:

Satisfied

Not Satisfied

Failed

Partially Satisfied

Present

Absent

Pending

Unknown

Uncertain

Disputed

Not Yet Evaluated

or another explicitly governed state where required.

The vocabulary should remain semantically controlled.

Satisfied Condition State

A Condition may be represented as:

Satisfied

when the governed Evaluation Inputs sufficiently meet the defined Requirement.

Therefore:

Satisfied Condition ≠ Established Liability.

Satisfaction concerns the Condition only.

Not Satisfied Condition State

A Condition may be:

Not Satisfied

where the governed inputs sufficiently establish that the Requirement has not been met.

This should remain distinct from:

Unknown, Pending, and Disputed.

Therefore:

Not Satisfied $\neq$ Unknown.

Failed Condition State

A Failed Condition State may be used where the applicable governance logic distinguishes a Requirement failure from ordinary non-satisfaction.

Where used, its meaning must be explicit.

Therefore:

Failed Condition $\neq$ Absence of Liability under every possible Basis.

Another Basis or conditional path may remain relevant.

Partially Satisfied Condition State

A compound or divisible Requirement may be only partially satisfied.

LCNSM should preserve which components are satisfied and which remain unsatisfied.

Therefore:

Partial Satisfaction $\neq$ Full Satisfaction.

Present Condition State

A Condition may require determination of whether a defined state is present.

Where sufficient input establishes presence, LCNSM may represent:

Present

Presence is not automatically equivalent to Condition Satisfaction unless the Requirement defines presence as sufficient.

Therefore:

Condition Presence $\neq$ Condition Satisfaction by default.

Absent Condition State

A Condition may require determination of whether a defined state is absent.

Where sufficient input establishes absence, LCNSM may represent:

Absent

Absence is not automatically failure.

For a Negative Requirement, absence may satisfy the Condition.

Therefore:

Condition Absence $\neq$ Condition Failure by default.

Pending Condition State

A Condition is Pending where evaluation cannot yet be completed because a required event, input, dependency, time period, or governance process remains outstanding.

Therefore:

Pending $\neq$ Not Satisfied.

Pending means the evaluation is not yet complete.

Unknown Condition State

A Condition is Unknown where available governed information is insufficient to determine its State reliably.

Therefore:

Unknown $\neq$ Satisfied.

Unknown $\neq$ Not Satisfied.

Unknown $\neq$ Failed.

Unknown must remain a legitimate governance state.

Uncertain Condition State

A Condition is Uncertain where available information permits evaluation but does not support a sufficiently reliable single State.

Uncertainty may arise from:

incomplete inputs, ambiguous inputs, measurement uncertainty, Requirement uncertainty, temporal uncertainty, or conflicting interpretation.

Therefore:

Uncertainty $\neq$ Authority to Manufacture Certainty.

Disputed Condition State

A Condition may be Disputed where materially relevant governed sources support competing State conclusions.

LCNSM should preserve:

the competing states, supporting inputs, sources, and unresolved conflict.

Therefore:

Disputed Condition $\neq$ Automatically Satisfied or Failed Condition.

Not Yet Evaluated State

A Condition may be identified and have a defined Requirement while no evaluation has yet occurred.

The State:

Not Yet Evaluated

should remain distinguishable from:

Unknown.

Unknown indicates attempted or relevant evaluation cannot establish a reliable State.

Not Yet Evaluated indicates that State evaluation has not yet been completed.

Positive Condition State

A Positive Requirement asks whether a required state exists, occurred, or was present.

LCNSM evaluates the relevant input against that Requirement.

Therefore:

Required Presence $\neq$ Established Presence.

Negative Condition State

A Negative Requirement asks whether a defined state was absent or did not occur.

The architecture must not infer absence solely from missing evidence.

Therefore:

Absence of Evidence $\neq$ Established Absence by default.

Threshold Condition State

A Threshold Requirement should be evaluated against the defined threshold logic.

The State should preserve:

the relevant value, the threshold, the comparison rule, and resulting Condition State where appropriate.

Therefore:

Threshold Crossing $\neq$ Liability Determination.

Threshold Uncertainty

Where the measured value or threshold is uncertain, the resulting Condition State should preserve that uncertainty.

A system must not convert approximate input into false threshold certainty.

Temporal Condition State

A Temporal Requirement must be evaluated against the relevant time.

The State should correspond to the required lifecycle moment.

Therefore:

Current Condition State $\neq$ Historical Condition State by default.

Historical State Evaluation

Where a Liability inquiry concerns time T1, LCNSM should evaluate the Condition using inputs applicable to T1.

Later states at T2 must not automatically replace them.

Therefore:

State at T2 $\neq$ State at T1 by default.

Duration State

Where the Requirement concerns duration, LCNSM should determine whether the required state persisted for the defined period.

Therefore:

Momentary State $\neq$ Duration Requirement Satisfaction.

Sequential Condition State

Where Requirements operate sequentially, LCNSM should preserve whether:

each required step occurred, the required order was maintained, and the sequence completed.

Therefore:

All Components Present $\neq$ Sequence Satisfied by default.

Order may itself be material.

Concurrent Condition State

Where multiple Requirements must be satisfied concurrently, LCNSM should preserve the State of each component and the resulting concurrent State.

Therefore:

One Satisfied Component $\neq$ Concurrent Condition Satisfaction.

Alternative Condition State

Where a Condition permits alternative paths, LCNSM should evaluate the permitted alternatives according to the governed Requirement logic.

Therefore:

Failure of Alternative A $\neq$ Failure of Condition if valid Alternative B is satisfied.

Cumulative Condition State

Where Requirements accumulate, LCNSM should preserve individual component States and the resulting cumulative State.

Therefore:

Individual Component State $\neq$ Cumulative Condition State.

Dependent Condition State

Where Condition B depends upon Condition A, LCNSM should preserve the dependency.

If the dependency is not activated, B should not automatically be treated as failed.

Therefore:

Inactive Dependency $\neq$ Failed Dependent Condition.

Nested Condition State

Where Requirement logic is nested, the resulting State should preserve that structure.

Example:

A AND (B OR C)

must be evaluated according to the actual nested logic.

Therefore:

Nested State Logic $\neq$ Flat State Aggregation.

Subject-Specific Condition State

A Condition may have different States for different Subjects.

Therefore:

Condition State for Subject A $\neq$ Condition State for Subject B.

A multi-party Liability inquiry should not collapse these States.

Relationship-Specific Condition State

The same Subject may have different Condition States in different Liability Relationships.

Therefore:

Condition State in Relationship A $\neq$ Condition State in Relationship B.

Object-Specific Condition State

The same Condition concept may produce different States for different Liability Objects.

Therefore:

Condition State for Object A $\neq$ Condition State for Object B.

Basis-Specific Condition State

The same fact may satisfy a Requirement under one Liability Basis while failing to satisfy another Basis's Requirement.

Therefore:

Condition State under Basis A $\neq$ Condition State under Basis B by default.

Multi-Basis Evaluation

Where multiple Liability Bases exist, LCNSM should preserve separate Condition State structures where their Requirements differ.

Therefore:

One Fact Pattern $\neq$ One Universal Condition State across all Bases.

State Confidence

A Condition State may include a governed confidence or reliability representation where methodologically necessary.

Confidence should reflect the quality and completeness of the evaluation inputs and logic.

It must not replace the State itself.

Therefore:

State Confidence $\neq$ Condition State.

High Confidence

A high-confidence Condition State may indicate strong governed support for the State.

It does not increase or decrease Liability by itself.

Therefore:

High Confidence $\neq$ Strong Liability.

Low Confidence

A low-confidence State may require:

additional input, reassessment, or preservation of uncertainty.

It should not be silently treated as definitive.

State Conflict

Different evaluation paths or governed inputs may produce conflicting Condition States.

LCNSM should preserve the conflict until it is sufficiently resolved.

Therefore:

State Conflict $\neq$ Authority to Select the Preferred State.

State Reconciliation

Where conflicting inputs can be reconciled through governed evaluation, LCNSM may establish a revised Condition State.

The reconciliation should remain traceable.

It must not erase the earlier conflict.

State Assumption

An assumed Condition State should never be represented as an established State without explicit qualification.

Therefore:

Condition State Assumption $\neq$ Established Condition State.

Assumptions must remain visible.

State Inference

Some Condition States may require inference from governed inputs.

Where inference is permitted, the system should preserve:

input basis, inference logic, uncertainty, and resulting State.

Therefore:

Inferred State $\neq$ Directly Observed State.

Direct and Inferred State

Directly established and inferred States may both be governable.

They should remain distinguishable where the difference is material.

This supports later Connection and Determination governance.

State Aggregation

Where a Condition contains multiple Requirement components, LCNSM may need to derive an aggregate Condition State.

Aggregation logic should be explicit.

Therefore:

Aggregate Condition State $\neq$ Arbitrary Summary of Component States.

State Granularity

The State should be represented at the level necessary to preserve material governance distinctions.

Excessive simplification may hide:

partial satisfaction, Subject differences, temporal differences, or Requirement-component differences.

However, State structures should not be expanded without governance need.

State Correction

A previously determined Condition State may later be found incorrect.

Correction should preserve:

the earlier State, the corrected State, the correction reason, supporting inputs, and relevant time.

Therefore:

Condition State Correction $\neq$ Historical Erasure.

State Reassessment

New evidence, corrected inputs, changed Requirement interpretation, or lifecycle re-entry may require Condition State reassessment.

Therefore:

Condition State Reassessment $\neq$ Historical Rewrite.

The earlier governed State remains part of the record.

State Change

A Condition State may legitimately change because underlying reality changes.

Example:

Pending $\rightarrow$ Satisfied Satisfied $\rightarrow$ Not Satisfied Unknown $\rightarrow$ Satisfied The architecture should distinguish:

State correction, from:

actual State change over time.

Therefore:

Condition State Correction $\neq$ Condition State Change.

State Change and Liability

A Condition State change may affect downstream Liability governance.

It does not automatically change Liability.

Therefore:

Condition State Change $\neq$ Liability Change by default.

Affected downstream stages may require reassessment.

State at Determination Time

Where Liability Determination later occurs, the relevant Condition State Set should be identifiable for the applicable Determination moment.

This supports traceability between:

Condition governance, Connection governance, and Liability Determination.

It does not merge those stages.

Condition State and Exception Governance

LCNSM determines the ordinary governed State of the Condition against its Requirement.

LCNXM subsequently determines whether an Exception, Exclusion, Waiver, or other modifying structure changes how that State operates.

Therefore:

Condition State $\neq$ Exception-Adjusted Condition State by default.

Satisfied State and Exception

A Condition may be satisfied while an applicable Exception modifies its effect.

Therefore:

Satisfied Condition $\neq$ Unmodified Conditional Effect by default.

LCNXM owns that analysis.

Failed State and Exception

A Condition may appear failed while an Exception or alternative permitted structure changes its downstream significance.

Therefore:

Failed Condition $\neq$ Final Conditional Outcome by default.

Condition State and Liability Connection

LCNSM does not determine Liability-Relevant Connection.

A satisfied Condition may exist without the required Connection.

Therefore:

Satisfied Condition $\neq$ Liability-Relevant Connection.

Connection and Condition State

A strong operational or factual connection may exist while a required Condition remains unsatisfied.

Therefore:

Liability-Relevant Connection $\neq$ Satisfied Condition.

The two governance objects remain separate.

Condition State and Liability Attachment

Condition State does not establish Liability Attachment.

Therefore:

Satisfied Condition $\neq$ Liability Attachment.

Attachment belongs to LDS.

Condition State and Liability Determination

Even where every currently identified Condition is satisfied:

Satisfied Condition Set $\neq$ Liability Determination.

Connection and Determination remain separate stages.

Condition State and Liability Scope

LCNSM does not determine how far Liability extends.

Therefore:

Condition State $\neq$ Liability Scope.

Condition State and Liability Allocation

LCNSM does not determine how Liability is distributed among Subjects.

Therefore:

Condition State $\neq$ Liability Allocation.

Subject-specific Condition States may later inform allocation without determining it.

Harm State and Condition State

Where a Condition depends on Harm, HAGA™ may provide the governed Harm State.

LCNSM evaluates that input against the Requirement.

Therefore:

Harm State $\neq$ Liability Condition State by itself.

Accountability State and Condition State

Where a Condition depends on Responsibility, Accountability, Authority, or Attribution, AGA™ may provide the relevant governed state.

Therefore:

Accountability State $\neq$ Liability Condition State by itself.

Identity State and Condition State

Where Identity, Ownership, Authority, Permission, Provenance, Transfer, or Continuity matters, OBIDENITY® may provide the relevant state.

LCNSM evaluates its conditional relevance.

Therefore:

Identity State $\neq$ Liability Condition State by itself.

AI or Autonomous-System State

AIG® or ASGA™ may provide relevant AI or autonomous-system governance states.

These may become Evaluation Inputs.

Therefore:

AI Governance State $\neq$ Liability Condition State by itself.

No Outcome-to-State Shortcut

LCNSM rejects:

Adverse Outcome → Assume Condition Satisfied and:

No Adverse Outcome → Assume Condition Not Satisfied The State must derive from the Requirement and relevant inputs.

Therefore:

Outcome ≠ Condition State.

No Harm-to-State Shortcut

The existence of Harm does not establish every Harm-related or Liability-related Condition.

Therefore:

Established Harm ≠ Satisfied Liability Condition by default.

No Breach-to-State Shortcut

A Breach label does not automatically establish the State of every Condition associated with an obligation.

Therefore:

Breach ≠ Condition State by default.

No Fault-to-State Shortcut

A Fault assertion does not establish a Condition State.

Therefore:

Fault Assertion ≠ Satisfied Fault-Related Condition.

The applicable Requirement must be evaluated.

No Relationship-to-State Shortcut

A Relationship may make a Condition relevant.

It does not establish the Condition State.

Therefore:

Liability Relationship ≠ Liability Condition State.

No Evidence-to-State Shortcut

A piece of evidence should not automatically be converted into a Condition State without applying the Requirement.

Therefore:

Evidence ≠ Condition State.

No Claim-to-State Shortcut

A Claim may assert that a Condition was satisfied or failed.

The assertion does not determine the State.

Therefore:

Claimed Condition State ≠ Governed Condition State.

State Sufficiency

A Condition State is sufficiently governed when the system can determine:

which Condition is being evaluated, which Requirement applies, which inputs were used, which temporal state applies, how the evaluation was performed, what State resulted, what uncertainty or conflict remains, and whether the State is sufficiently reliable for Exception governance.

State Set Sufficiency

The Governed Liability Condition State Set is sufficient when materially relevant Conditions have an explicit State or an explicit unresolved State.

Sufficiency does not require every Condition to be Satisfied.

Therefore:

State Governance Sufficiency $\neq$ Condition Satisfaction.

Unknown, Pending, Uncertain, or Disputed States may remain valid outputs where correctly governed.

Handoff to Liability Condition Exception Governance

Once the Condition States are sufficiently governed, LCNSM produces:

Governed Liability Condition State Set

The handoff should preserve:

Condition Identity, Requirement Identity, Basis reference, Subject reference, Relationship reference, Object reference, Evaluation Inputs, relevant time, Condition State, component States, threshold results, temporal results, confidence where used, uncertainty, conflicts, assumptions, inferences, and State correction or reassessment history where relevant.

The next governance question becomes:

Do any Exceptions, Exclusions, Waivers, or other governed modifying structures alter the applicability or effect of the established Liability Condition States?

That question belongs to:

LCNXM — Liability Condition Exception Module.

Minimum Implementation Framework

A conforming implementation of LCNSM must provide at least the following eight capabilities:

Evaluation Input Association

Capability to associate each Condition Requirement with the materially relevant governed inputs, their sources, temporal applicability, and traceability.

Requirement-to-State Evaluation

Capability to evaluate governed inputs against explicit Condition Requirements without substituting outcome, Relationship, Claim, Harm, Fault, or evidence for the evaluation itself.

Explicit Condition State Governance

Capability to represent Satisfied, Not Satisfied, Failed, Partially Satisfied, Present, Absent, Pending, Unknown, Uncertain, Disputed, Not Yet Evaluated, and other required governed States.

Threshold, Temporal & Compound State Governance

Capability to evaluate threshold, temporal, duration, sequential, concurrent, alternative, cumulative, dependent, conditional, and nested Requirement structures.

Subject, Relationship, Object & Basis-Specific State Governance

Capability to preserve materially distinct Condition States across Subjects, Relationships, Objects, and Liability Bases without inappropriate state collapse.

Confidence, Uncertainty, Conflict & Inference Governance

Capability to preserve State confidence where used, uncertainty, disputes, conflicting inputs, assumptions, and inferred States without manufacturing certainty.

State Change, Correction & Reassessment Governance

Capability to distinguish actual Condition State change from correction or reassessment and to preserve earlier governed States without historical erasure.

State Sufficiency & Handoff

Capability to determine whether the Condition State Set is sufficiently governed for Liability Condition Exception governance while preserving unresolved States explicitly.

Governance Logic

The canonical LCNSM governance logic is:

Governed Liability Condition Requirement Set

↓

Associate Relevant Evaluation Inputs

↓

Verify Temporal Applicability

↓

Evaluate Input Against Requirement

↓ Evaluate Threshold / Temporal / Compound Logic where applicable ↓

Determine Component States

↓

Determine Condition State

↓

Preserve Subject / Relationship / Object / Basis Specificity

↓ Assess Confidence where applicable ↓

Preserve Unknown / Pending / Uncertain / Disputed States

↓

Preserve Conflict / Assumption / Inference

↓

Record State Change / Correction / Reassessment

↓

Assess State Sufficiency

↓

Governed Liability Condition State Set

↓

Handoff to Liability Condition Exception Governance

Governance Outputs

LCNSM may produce:

Liability Condition State
Satisfied Condition State
Not Satisfied Condition State
Failed Condition State
Partially Satisfied Condition State
Present Condition State
Absent Condition State
Pending Condition State
Unknown Condition State
Uncertain Condition State
Disputed Condition State
Not Yet Evaluated Condition State
Positive Condition State
Negative Condition State
Threshold Condition State
Temporal Condition State
Duration Condition State
Sequential Condition State
Concurrent Condition State
Alternative Condition State
Cumulative Condition State
Dependent Condition State
Nested Condition State
Subject-Specific Condition State
Relationship-Specific Condition State
Object-Specific Condition State
Basis-Specific Condition State
State Confidence Record
State Conflict Record
State Assumption Record

State Inference Record

State Correction Record

State Reassessment Record

State Change Record

State Sufficiency State

The principal output is:

Governed Liability Condition State Set

Critical Distinctions

LCNSM preserves the following distinctions:

Condition State ≠ Liability Determination Liability Condition State ≠ Liability State Condition State without Requirement ≠ Governed Condition State Evaluation Input ≠ Condition State Multiple Inputs ≠ Automatic Condition Certainty Missing Input ≠ Failed Condition by default Evidence ≠ Condition State Validated Input ≠ Satisfied Condition by default Operational Reality ≠ Condition State by itself Satisfied Condition ≠ Established Liability Not Satisfied ≠ Unknown Failed Condition ≠ Absence of Liability under every possible Basis Partial Satisfaction ≠ Full Satisfaction Condition Presence ≠ Condition Satisfaction by default Condition Absence ≠ Condition Failure by default Pending ≠ Not Satisfied Unknown ≠ Satisfied Unknown ≠ Not Satisfied Unknown ≠ Failed Uncertainty ≠ Authority to Manufacture Certainty Disputed Condition ≠ Automatically Satisfied or Failed Condition Not Yet Evaluated ≠ Unknown Required Presence ≠ Established Presence Absence of Evidence ≠ Established Absence by default Threshold Crossing ≠ Liability Determination Current Condition State ≠ Historical Condition State by default State at T2 ≠ State at T1 by default Momentary State ≠ Duration Requirement Satisfaction All Components Present ≠ Sequence Satisfied by default One Satisfied Component ≠ Concurrent Condition Satisfaction Failure of Alternative A ≠ Failure of Condition if valid Alternative B is satisfied Individual Component State ≠ Cumulative Condition State Inactive Dependency ≠ Failed Dependent Condition Nested State Logic ≠ Flat State Aggregation Condition State for Subject A ≠ Condition State for Subject B Condition State in Relationship A ≠ Condition State in Relationship B Condition State for Object A ≠ Condition State for Object B Condition State under Basis A ≠ Condition State under Basis B by default One Fact Pattern ≠ One Universal Condition State across all Bases State Confidence ≠ Condition State High Confidence ≠ Strong Liability State Conflict ≠ Authority to Select the Preferred State Condition State Assumption ≠ Established Condition State Inferred State ≠ Directly Observed State Aggregate Condition State ≠ Arbitrary Summary of Component States Condition State Correction ≠ Historical Erasure Condition State Reassessment ≠ Historical Rewrite Condition State Correction ≠ Condition State Change Condition State Change ≠ Liability Change by default Condition State ≠ Exception-Adjusted Condition State by default Satisfied Condition ≠ Unmodified Conditional Effect by default Failed Condition ≠ Final Conditional Outcome by default Satisfied Condition ≠ Liability-Relevant Connection Liability-Relevant Connection ≠ Satisfied Condition Satisfied Condition ≠ Liability Attachment Satisfied Condition Set ≠ Liability Determination Condition State ≠ Liability Scope Condition State ≠ Liability Allocation Harm State ≠ Liability Condition State by itself Accountability State ≠ Liability Condition State by itself Identity State ≠ Liability Condition State by itself AI Governance State ≠ Liability Condition State by itself Outcome ≠ Condition State Established Harm ≠ Satisfied Liability Condition by default Breach ≠ Condition State by default Fault Assertion ≠ Satisfied Fault-Related Condition Liability Relationship ≠ Liability Condition State Claimed Condition State ≠ Governed Condition State State Governance Sufficiency ≠ Condition Satisfaction

Cross-Architecture Boundary

LCNSM may consume governed inputs from other OOF® architectures where those inputs are required to evaluate a Liability Condition.

GOA™ may provide Authority, Scope, and Delegation states.

ORA™ may provide Operational Reality.

AGA™ may provide Responsibility, Accountability, Authority, and Attribution states.

HAGA™ may provide governed Harm states.

ADIT® may provide evidence and auditability.

VALIDOS™ may provide validated inputs.

OBIDENTITY® may provide Identity, Ownership, Authority, Permission, Provenance, Transfer, and Continuity states.

AIG® may provide AI governance states.

ASGA™ may provide autonomous-system governance and operational states.

LCNSM evaluates these inputs only against the applicable Liability Condition Requirement.

The canonical interoperability principle is:

An externally governed state may determine or inform a Liability Condition State without becoming Liability and without transferring governance ownership of its original domain to LIGA™.

Canonical Boundary

LCNSM begins with a:

Governed Liability Condition Requirement Set

and asks:

What is the governed state of each Condition when the relevant inputs are evaluated against its Requirement?

LCNSM governs:

Evaluation Input Association, Requirement Evaluation, Condition State Determination, Satisfied State, Not Satisfied State, Failed State, Partial Satisfaction, Presence, Absence, Pending State, Unknown State, Uncertain State, Disputed State, Threshold State, Temporal State, Duration State, Compound State, Subject-Specific State, Relationship-Specific State, Object-Specific State, Basis-Specific State, State Confidence, State Conflict, State Assumption, State Inference, State Change, State Correction, State Reassessment, and State Sufficiency.

LCNSM does not determine:

Exception Applicability, Exclusion Applicability, Waiver Effect, Liability-Relevant Connection, Liability Attachment, Liability Determination, Liability Scope, Liability Allocation, Liability State, Liability Transfer, Recourse, Resolution, Discharge, or Finality.

LCNSM ends when a:

Governed Liability Condition State Set

is sufficiently established for Liability Condition Exception governance.

Use Case 1 — AI Oversight Condition State

LCNRM has defined a Requirement that a designated human operator must provide a specified form of oversight during a defined AI-enabled operational process.

LCNSM receives governed operational records, authority information, and relevant evidence.

It evaluates whether the required oversight occurred during the applicable period.

The resulting Condition State may be:

Satisfied, Not Satisfied, Partially Satisfied, Unknown, or Disputed.

LCNSM does not determine whether failure of the oversight Condition creates Liability.

It determines only the governed State of that Condition.

Use Case 2 — Industrial Maintenance Condition State

LCNRM has defined a Requirement that specified maintenance must occur for a particular equipment Object within a defined interval.

LCNSM evaluates maintenance records, operational history, timing, and other relevant governed inputs against that Requirement.

If the required maintenance is established within the applicable period, the Condition may be represented as Satisfied.

If the available records are incomplete, the appropriate State may instead be Unknown or Disputed.

Equipment failure itself does not determine the maintenance Condition State.

Canonical Closing Statement

A Liability Condition becomes governable only when its Requirement is evaluated against the relevant reality rather than inferred from the outcome that followed.

LCNSM preserves satisfied, unsatisfied, partial, unknown, uncertain, disputed, and temporal states without converting any of them directly into Liability.

The Requirement defines the test. The Condition State records the result. Liability remains a later determination.

Determine the Condition State before governing its Exceptions and downstream Liability relevance.

Governed Links

Liability Conditions Standard (LCOS)

LIGA™ Architecture Map

LCNXM — Liability Condition Exception Module

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the MIP® Protection & Licensing terms or contact OOF® directly.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/content/liga/liga-lcos-lcnsm-3.html>

OOF® · MIP® Protected · Free permitted use · Commercial/derivative use may require authorization · Active Protection

Canonical source: <https://originopenfoundation.org/content/liga/liga-lcos-lcnsm-3.html>